

Python Lab Task

Conditional Statements • Iteration Statements • Break, Continue, Pass • Pattern Programs

Instructions: Write, run, and test each program in a Python IDE/interpreter. For every exercise, include your source code and a screenshot/output in your lab record. No solutions are provided in this sheet — you are expected to design and write the logic yourself.

Part 1: Conditional Statements

Conditional statements let a program choose between different paths of execution based on whether a condition is **True** or **False**.

1.1 The if Statement

```
if condition:
    statement(s)
```

1.2 if-else Statement

```
if condition:
    statement(s)
else:
    statement(s)
```

1.3 if-elif-else Ladder

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)
```

1.4 Nested if

```
if condition1:
    if condition2:
        statement(s)
```

Exercises – Conditional Statements

Exercise 1: Write a program to check whether a given number is positive, negative, or zero.

Exercise 2: Write a program to check whether a given year is a leap year or not.

Exercise 3: Write a program that accepts three sides of a triangle and determines whether it is equilateral, isosceles, scalene, or not a valid triangle at all.

Exercise 4: Write a program to find the largest of three numbers using nested if-else.

Exercise 5: Write a program that takes a student's marks and prints the grade using if-elif-else (A: ≥ 90 , B: 75–89, C: 60–74, D: 40–59, F: below 40).

Exercise 6: Write a program to check whether a character entered by the user is a vowel, consonant, digit, or special symbol.

Exercise 7: Write a program that accepts a year, month, and day, and determines whether the date is valid, accounting for leap years and the number of days in each month.

Part 2: Iteration Statements: while and for

2.1 while Loop

Repeats a block of code as long as a condition remains True.

```
while condition:
    statement(s)
    update_expression
```

2.2 for Loop

Iterates over a sequence (list, string, range, tuple, etc.).

```
for variable in sequence:
    statement(s)

for i in range(start, stop, step):
    statement(s)
```

Exercises – while Loop

Exercise 8: Write a program to print the first N natural numbers using a while loop.

Exercise 9: Write a program to find the sum and average of digits of a given number using a while loop.

Exercise 10: Write a program to reverse a given integer using a while loop (e.g., 1234 $\rightarrow$ 4321).

Exercise 11: Write a program to check whether a given number is a palindrome using a while loop.

Exercise 12: Write a program to generate the Fibonacci series up to N terms using a while loop.

Exercises – for Loop

Exercise 13: Write a program to print the multiplication table of a given number using a for loop.

Exercise 14: Write a program to find the factorial of a given number using a for loop.

Exercise 15: Write a program to count the number of vowels, consonants, digits, and spaces in a given string.

Exercise 16: Write a program to check whether a given number is prime using a for loop.

Exercise 17: Write a program to print all prime numbers between two given limits.

Part 3: Pattern Printing Programs

Use nested for/while loops (combined with conditional logic where needed) to print each of the following patterns for a given number of rows, N. Take N as input from the user unless stated otherwise.

Exercise 18: Print a right-angled triangle of stars (N = 5):

```
*
* *
* * *
* * * *
* * * * *
```

Exercise 19: Print an inverted right-angled triangle of stars (N = 5):

```
* * * * *
* * * *
* * *
* *
*
```

Exercise 20: Print a pyramid (centered triangle) of stars (N = 5):

```
  *
 * *
* * *
* * * *
* * * * *
```

Exercise 21: Print an inverted pyramid of stars (N = 5):

```
* * * * *
* * * *
* * *
* *
*
```

Exercise 22: Print a diamond pattern using stars (N = 4, i.e. 2N rows total):

```
  *
 ***
*****
*****
*****
 *****
  ***
   *
```

Exercise 23: Print a number triangle where each row repeats its row number (N = 5):

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Exercise 24: Print a number triangle with increasing sequence in each row (N = 5):

```
1
1 2
1 2 3
1 2 3 4
```

1 2 3 4 5

Exercise 25: Print Pascal's-Triangle-style pattern using increasing then decreasing numbers (N = 5):

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

Exercise 26: Print the alphabet pattern below (N = 5, using uppercase letters):

```
A
B B
C C C
D D D D
E E E E E
```

Exercise 27: Print a hollow square pattern of stars (N = 5):

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Exercise 28: Print a hollow diamond pattern of stars (N = 4):

```
  *
 * *
*   *
*   *
 *   *
  * *
   *
```

Exercise 29: Print a floyd's triangle (N = 5 rows, consecutive natural numbers):

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Exercise 30: Print a butterfly pattern of stars (N = 4):

```
*           *
* *       * *
* * *   * * *
* * * ** * * *
* * *   * * *
* *       * *
*           *
```

Tip: Most patterns need two nested loops — an outer loop for rows and an inner loop (or two) for columns/spaces. Think separately about the logic for spaces and the logic for symbols/numbers in each row.